



Last chance to fix `std::nontype`

Document number:

[P3740R0](#)

Date:

2025-06-14

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3740/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/nontype.cow



ABSTRACT: P2472R3 introduced overloads for the `std::function_ref` constructor which utilize a helper type `std::nontype`. This helper type may be obsoleted by other active proposals, and its name has become a bad choice now that C++26 no longer replaced the term "non-type template parameter", with "constant template parameter".

§ Contents

- 1 **Introduction**
 - 1.1 Why did we need a new `std::nontype` type?
 - 1.2 Refresher on what `std::nontype` accomplishes
- 2 **Proposed solution**
 - 2.1 Comparison with `std::nontype`
 - 2.1.1 Note on free function ergonomics
 - 2.2 Possible misuse of `std::stateless`
 - 2.3 Advantages compared to `std::nontype`
- 3 **Alternatives considered**
 - 3.1 Removing `std::nontype` with no replacement, revisiting for C++29
 - 3.2 Waiting for `constexpr` function parameters
 - 3.3 Replacing `std::nontype` with `std::constant_wrapper`
 - 3.4 Exposing the *BoundEntityType*
 - 3.5 Renaming `std::nontype`
 - 3.6 Conclusion



4	Implementation experience
5	Wording
5.1	Option A — Replacing <code>std::nontype</code> with a <code>std::stateless</code> tag
5.2	Option B — Renaming <code>std::nontype</code> to <code>std::constant_arg</code>
6	References

§ 1. Introduction

[P2472R3] proposed additional overloads for the `std::function_ref` constructor which utilize a helper type `std::nontype`. This paper was merged into [P0792R14], which was plenary-approved at Varna 2023 for C++26.

There is now a proposal [P2781R8] on track for C++29 which adds a `std::constant_wrapper` helper type, and this could be a plausible replacement for `std::nontype`. Considering that, do we even need `std::nontype`? Furthermore, the naming choice `std::nontype` makes very little sense following [P2841R1], which introduced concept and variable template parameters. Since those are not types either, what the standard used to call "non-type template parameter" has been renamed to "constant template parameter".

In short, we now have to decide *for* C++26 whether `std::nontype` should be renamed (and if so, what to), whether it should be merged with `std::constant_wrapper`, or whether these constructor overloads should be delayed until C++29.



NOTE: The class template is called `std::nontype_t`, and the corresponding variable template is called `std::nontype`. This document refers to the feature as a whole as `std::nontype`.

§ 1.1. Why did we need a new `std::nontype` type?

An obvious question may be why the existing `std::integral_constant` cannot be used instead. This has multiple reasons:

- `std::integral_constant<class T, T v>` requires specifying the type separately.
- `std::integral_constant` has a lengthy name.
- `std::integral_constant` has a call operator which returns `v`, which produces friction with the other constructors.

§ 1.2. Refresher on what `std::nontype` accomplishes

`std::nontype` is used only within constructors of `std::function_ref`:

```
// constexpr and noexcept omitted for simplicity
template<class F>      function_ref(F*);
template<class F>      function_ref(F&&);
template<auto f>       function_ref(nontype_t<f>);
template<auto f, class U> function_ref(nontype_t<f>, U&&);
template<auto f, class T> function_ref(nontype_t<f>, cv T*);
```



Intuitively, `std::function_ref` is the C++ counterpart to the C idiom of passing `void*` and a function pointer which that `void*` is fed into, as seen in `qsort`. It is extremely common practice to provide a null pointer to `qsort` and thus rely on a "capture-less" comparison.

With `std::nontype`, `std::function_ref` can support such use "capture-less" uses:

```
// once again, some simplifications (no noexcept, no invoke, etc.) ...
template<auto f>
function_ref(nontype_t<f>) {
    // bound-entity is similar to the void* we would have stored in C.
    // We can leave it empty since we are constructing from some global callable.
    this->bound-entity = {};
    // Note that lambdas with no captures can be converted to function pointers.
    this->thunk_ptr = [] (BoundEntityType, Args&&... args) -> R {
        // Notice that we don't need to capture f.
        // This is only possible because f is a constant template parameter.
        return f(std::forward<Args&&>(args)...);
    };
}
```

Crucially, it would be impossible to create a `std::function_ref` from a function directly without this helper. At best, we could store function pointer within *bound-entity* and call that function pointer from within *bound-entity*. This would introduce entirely unnecessary overhead.

§ 2. Proposed solution

We can ditch `std::nontype` and obtain the functionality it provides in a different way. This would be very tempting because we don't have the dilemma of naming it right, making it redundant with `std::constant_wrapper`, having a weird workaround that becomes obsolete with `constexpr` function parameters, etc. Instead, we can get the same functionality using a tagged constructor.

The crucial observation in §1.2. [Refresher on what std::nontype accomplishes](#) is that the lambda we create for *thunk_ptr* cannot capture the constructor parameters. However, that is not actually necessary if we only need the type of the parameter, not its identity. The same approach is used for `std::less`, `std::allocator`, `std::hash`, etc. We don't need to hold an instance of these; we can just default-construct when needed. Specifically, the tagged constructor would function like:

```
// Helper tag type.
struct stateless_t { };
inline constexpr stateless_t stateless;

// once again, some simplifications (no noexcept, no invoke, etc.) ...
template<class F, class T>
function_ref(stateless_t, const F&, cv T* obj) {
    // In practice, this may be slightly more complicated, if BoundEntityType is not ju
    this->bound-entity = obj;

    this->thunk_ptr = [] (BoundEntityType bound, Args&&... args) -> R {
        auto obj = static_cast<cv T*>(bound);
        // No captures needed because we construct a new T here.
        return F{}(obj, std::forward<Args&&>(args)...);
    };
}
```



```
// Same idea, but bound-entity becomes null:
template<class F> function_ref(stateless_t, const F&);
// Same idea, but bound-entity becomes addressof(obj):
template<class F, class U> function_ref(stateless_t, U&& obj);
```

§ 2.1. Comparison with `std::nontype`

<code>std::nontype</code>	<code>std::stateless</code>
<pre>// construction from free function int func(int); std::function_ref<int(int)> r = std::nontype<func>; //</pre>	<pre>// construction from free function int func(int); std::function_ref<int(int)> r = { std::stateless, [](int x) { return func(x); } };</pre>
<pre>// construction from member struct S { int x; }; std::function_ref<int(S)> r = std::nontype<&S::x>; //</pre>	<pre>// construction from member struct S { int x; }; std::function_ref<int(S)> r = { std::stateless, [](S s) { return s.x; } };</pre>
<pre>// construction from stateless type std::function_ref<bool(int,int)> r = std::nontype<std::less<int>{}>;</pre>	<pre>// construction from stateless type std::function_ref<bool(int,int)> r = { std::stateless, std::less<int>{} };</pre>
<pre>// if we need a lambda anyway ... std::vector<int> out = /* ... */; constexpr auto put = [](auto& o, int x) { o.push_back(x); }; std::function_ref<void(int)> consumer = { std::nontype<put>, out }; //</pre>	<pre>// if we need a lambda anyway ... std::vector<int> out = /* ... */; constexpr auto put = [](auto& o, int x) { o.push_back(x); }; std::function_ref<void(int)> consumer { std::stateless, put, out }; //</pre>

§ 2.1.1. Note on free function ergonomics

While the construction from a regular function seems much more verbose with tagged constructors, [\[P3312R1\]](#) "Overload Set Types" could make this substantially more concise. Furthermore, it is common practice to create `LIFT` macros for this purpose, which would be used like:

```
std::function_ref<int(int)> r = { std::stateless, LIFT(func) };
```

Since most standard library functions aren't addressable anyway, the ergonomics of creating a `std::function_ref` directly from a free function may not be that significant. In fact, it could be argued that `std::nontype` encourages the user to write possibly-ill-formed code like `std::nontype<std::sqrtf>` by making it so frictionless.

§ 2.2. Possible misuse of `std::stateless`



BUG: It is possible to misuse the proposed feature as follows:

```
int f(int);
std::function_ref<int(int)> r = { std::stateless, f };
```

This would invoke a value-constructed `int(*)()` when `r` is called, resulting in undefined behavior.

This possible bug can be easily caught by adding deleted constructors though:

```
function_ref(stateless_t, auto*) = delete;
function_ref(stateless_t, auto*, auto&&) = delete;
```

An even better option would be to add *Mandates* specifications to the `stateless_t` constructors which require that the given callable is not a function pointer type. Although none of these fixes prevent the user from writing this mistake, it can be caught statically with ease, rather than actually resulting in undefined behavior.

§ 2.3. Advantages compared to `std::nontype`

The proposed approach has several advantages:

- Providing tag types within constructors is a more well-established design within the standard library.
- The naming issue for `std::nontype` is eliminated completely. While the tag also needs a good name, we don't need to worry about conflicting with `std::constant_wrapper`, which seems harder to solve.
- Potential user confusion over what the difference between `std::nontype` and `std::constant_wrapper` is, and why the language needs both is, is eliminated.
- The design wouldn't feel immediately obsolete and deprecation-worthy once/if we get `constexpr` function parameters ([P1045R1]). It is worth noting that `std::stateless` does not require structural types, so there may be niche use cases where it could be used and `std::nontype` couldn't.

§ 3. Alternatives considered

Besides the proposed approach, there are other possible solutions. However, the author position is that every one of them is worse than what is proposed.

§ 3.1. Removing `std::nontype` with no replacement, revisiting for C++29

Since there is a lot of active work in this area, perhaps we could simply shove `std::nontype` into C++29 and deal with the problem later. However, as demonstrated in §1.2. [Refresher on what `std::nontype` accomplishes](#), this helper type covers crucial use cases such as

- constructing `std::function_ref` from a free function with no overhead, or

- constructing `std::function_ref` from an existing function pointer and `void*`, in the style of `qsort`.



Author position: The functionality provided by `std::nontype` is a crucial part of `std::function_ref`, not a nice-to-have feature that can be ripped out. `std::function_ref` without these constructors would be akin to using containers without move semantics.

§ 3.2. Waiting for `constexpr` function parameters

`std::integral_constant`, `std::constant_wrapper`, and `std::nontype` are – to an extent – temporary hacks. If we were able to write

```
function_ref constexpr F*);
```

... then the workaround of `std::nontype` would be obsolete.

At Kona 2023, EWG showed enthusiasm for this option when discussing [\[P1045R1\]](#) "constexpr Function parameters".

”

Poll: P2781R3 “`std::constexpr_v`” and P1045R1 “constexpr function parameters” EWG would like to solve the problem solved by `std::constexpr_v` in the language, for example as proposed by P1045R1 or with “expression aliases”, rather than solving it in library. An implementation is desired. C++26 seems ambitious.

SF	F	N	A	SA
6	8	5	1	0

However, [\[P1045R1\]](#) has not seen any activity from the author since 2019. As nice as a core language solution would be, standard library features cannot be built on hopes, dreams, and unimplemented hypotheticals. Even if `constexpr` function parameters were eventually implemented, it may be possible to integrate them into the existing set of constructors and to deprecate `std::nontype`.

Author position: Overall, there is little reason to treat `constexpr` function parameters as a prerequisite; we don't need to wait.

§ 3.3. Replacing `std::nontype` with `std::constant_wrapper`

The seemingly obvious solution is to use `std::constant_wrapper` from [\[P2781R8\]](#). The option to use `std::constant_wrapper` as a replacement for `std::nontype` was discussed during an LEWG telecon 2025-03-11.

Major concerns have been raised over the fact that `std::constant_wrapper` is substantially more complex than `std::nontype`:

- `std::constant_wrapper` has an `operator()` already. If we have, say, `std::cw<8f>` made from some `int f()`, calling this wrapper would give us a `std::constant_wrapper` convertible to `int`. This means that it could also be passed into the constructor of a `std::function_ref<int()>` taking some callable type `F&&`.

- `std::constant_wrapper` also has a user-defined conversion function which returns the constant it wraps. If it wraps a function pointer (and this is a desired use case for `std::nontype`), this would also make the `std::function_ref` constructor taking `F*` (i.e. the function pointer constructor) a candidate. However, that candidate would always lose in overload resolution, so this is not a major concern.
- `std::constant_wrapper` would only be treated specially by `std::function_ref`, but due to the behavior described in the previous bullets, it could also be passed into other "function wrappers" like `std::move_only_function`. This could easily confuse users who may expect that `std::function_ref` is a general mechanism for wrapping functions passed to such "function wrappers".

However, even with these major concerns, it would not be *impossible* to use `std::constant_wrapper` for this. The `F&&` and `F*` constructors should be losing in overload resolution anyway. Even if they didn't, the overload set could be modified to resolve ambiguity.

Author position: Considering the call operator and user-defined conversion functions of `std::function_wrapper`, the type is not a good fit for `std::function_ref` construction.

§ 3.4. Exposing the *BoundEntityType*

Remember the example in §1.2. [Refresher on what `std::nontype` accomplishes](#). Our end goal is to enable more possible constructions of `std::function_ref`. Technically, this could be done by ripping the implementation open and exposing its details:

```
// new constructor
function_ref(R(*thunk)(BoundEntityType, Args&&...), BoundEntityType entity = { })
: thunk_ptr(thunk)
, bound_entity(entity) { }
```

However, this means that the user directly interacts with *BoundEntityType*, and this is a detail which may vary from implementation to implementation. For implementations that support conversion between function pointers and `void*`, *BoundEntityType* could simply be `cv void*`, but this may also be a [union](#), it may be multiple pointers large, etc.

Author position: Exposing the *BoundEntityType* to the user seems like a bad idea, and it would encourage writing code that is inadvertently not portable.

§ 3.5. Renaming `std::nontype`

Perhaps the most simple solution is to simply rename `std::nontype`. However, no good candidate with high consensus has been presented yet, and many options would be awfully similar to `std::constant_wrapper`, creating confusion:

- `std::constant` seems almost like a "more official" alternative to `std::constant_wrapper`, when this type is extremely limited on scope.
- `std::ctp`, `std::cta`, `std::ct` and other abbreviations derived from "constant template argument/parameter" feel out-of-place in the standard library, and cannibalize `std::cw`, the variable template for `std::constant_wrapper`.
- `std::const_arg`, `std::constant_arg`, `std::constexpr_arg` and other such names describe the purpose of the type well, but are still reminiscent of `std::constant_wrapper`.

Author position: Renaming `std::nontype` is not trivial, and it's unlikely that any solution will make everyone happy. Of all the options, `std::constant_arg` seems like the least bad candidate.



§ 3.6. Conclusion

Every alternative seems bad in its own way, but §3.5. Renaming `std::nontype` seems least intrusive, and is definitely something that we could do for C++26.

In §5. Wording, both the wording for §2. Proposed solution and for §3.5. Renaming `std::nontype` is presented.

§ 4. Implementation experience

§2. Proposed solution has been implemented at [GitHub]. This was done by forking the existing reference implementation of [P0792R14].

No issues whatsoever were encountered in the process; either solution provides the same functionality for the most part, although `std::nontype` does provide more direct support for function pointers and member pointers.

§ 5. Wording

The following changes are relative to [N5008].

§ 5.1. Option A — Replacing `std::nontype` with a `std::stateless` tag

In [version.syn], update the feature-test macro:

```
 #define __cpp_lib_function_ref 202306L 20XXXXL // also in <functional>
```

In [utility.syn], delete the declarations of `nontype_t` and `nontype`.

```
 // nontype argument tag
template<auto V>
    struct nontype_t {
        explicit nontype_t() = default;
    };
template<auto V> constexpr nontype_t<V> nontype{};
```

In its place, insert the following declaration:

```
 // stateless argument tag
struct stateless_t {
    explicit stateless_t() = default;
};
inline constexpr stateless_t stateless{};
```

Change [func.wrap.ref.class], as follows:



```

namespace std {
    template<class R, class... ArgTypes>
    class function_ref<R(ArgTypes...) cv noexcept(noex)> {
    public:
        // [func.wrap.ref.ctor], constructors and assignment operators
        template<class F> function_ref(F*) noexcept;
        template<class F> constexpr function_ref(F&&) noexcept;
template<auto f> constexpr function_ref(nontype_t<f>) noexcept;
template<auto f, class U> constexpr function_ref(nontype_t<f>, U&&) noexcept
template<auto f, class T> constexpr function_ref(nontype_t<f>, cv T*) noexcept
        template<class F> constexpr function_ref(stateless_t, const F&) noexcept;
        template<class F, class U> constexpr function_ref(stateless_t, const F&, U&&)
        template<class F, class T> constexpr function_ref(stateless_t, const F&, cv

        constexpr function_ref(const function_ref&) noexcept = default;
        constexpr function_ref& operator=(const function_ref&) noexcept = default;
        template<class T> function_ref& operator=(T) = delete;

        // [func.wrap.ref.inv], invocation
        R operator()(ArgTypes...) const noexcept(noex);

    private:
        template<class... T>
            static constexpr bool is-invocable-using = see below; // exposition only

        R (*thunk_ptr)(BoundEntityType, Args&&...) noexcept(noex); // exposition only
        BoundEntityType bound-entity; // exposition only
    };

    // [func.wrap.ref.deduct], deduction guides
    template<class F>
        function_ref(F*) -> function_ref<F>;
template<auto f>
function_ref(nontype_t<f>) -> function_ref<see below>;
template<auto f, class T>
function_ref(nontype_t<f>, T&&) -> function_ref<see below>;
    }

```

Change [func.wrap.ref.ctor] as follows:



```

template<auto f class F> constexpr function_ref(nontype_t<f> stateless_t, const

9 Let F be decltype(f):

10 Constraints: is-invocable-using<F> && is_default_constructible_v<F> is
true.

11 Mandates: If is_pointer_v<F> || is_member_pointer_v<F> is true, then f !=
nullptr is true. is_pointer_v<F> || is_member_pointer_v<F> is false.

12 Effects: Initializes bound-entity with a pointer to an unspecified object or null pointer
value, and thunk_ptr with the address of a function thunk such that thunk(bound-
entity, call-args...) is expression-equivalent ([defns.expression.equivalent]) to
invoke_r<R>(f F{}, call-args...).

```



```

template<auto-f class F, class U>
constexpr function_ref(nontype_t<f> stateless_t, const F&, U&& obj) noexcept;

13 Let T be remove_reference_t<U> and F be decltype(f).

14 Constraints:
    • is_rvalue_reference_v<U&&> is false, and
    • is_invocable_using<F, cv T&> is true, and
    • is_default_constructible_v<F> is true.

15 Mandates: If is_pointer_v<F> || is_member_pointer_v<F> is true, then f != nullptr is true. is_pointer_v<F> || is_member_pointer_v<F> is false.

16 Effects: Initializes bound-entity with addressof(obj), and thunk_ptr with the
address of a function thunk such that thunk(bound-entity, call-args...) is
expression-equivalent ([defns.expression.equivalent]) to invoke_r<R>(f F{},
static_cast<cv T&>(obj), call-args...).

template<auto-f class F, class T>
constexpr function_ref(nontype_t<f> stateless_t, const F&, cv T* obj) noexcept

17 Let F be decltype(f).

18 Constraints: is_invocable_using<F, cv T*> &&
is_default_constructible_v<F> is true.

19 Mandates: If is_pointer_v<F> || is_member_pointer_v<F> is true, then f != nullptr is true. is_pointer_v<F> || is_member_pointer_v<F> is false.

20 Preconditions: If is_member_pointer_v<F> is true, obj is not a null pointer.

21 Effects: Initializes bound-entity with obj, and thunk_ptr with the address of a
function thunk such that thunk(bound-entity, call-args...) is expression-
equivalent ([defns.expression.equivalent]) to invoke_r<R>(f F{}, obj, call-
args...).

```

In [func.wrap.ref.deduct], delete the following deduction guides:



```

template<auto f>
function_ref(nontype_t<f>) -> function_ref<@see below>;

2 Let F be remove_pointer_t<decltype(f)>.

3 Constraints: is_function_v<F> is true.

4 Remarks: The deduced type is function_ref<F>.

template<auto f, class T>
function_ref(nontype_t<f>, T&&) -> function_ref<see below>;

5 Let F be decltype(f).

6 Constraints:

```

- F is of the form `R(G::*)(A...) cv &opt noexcept(E)` for a type G, or
- F is of the form `M G::*` for a type G and an object type M, in which case let R be `invoke_result_t<F, T&>`, A... be an empty pack, and E be `false`, or
- F is of the form `R(*) (G, A...) noexcept(E)` for a type G.

7 Remarks: The deduced type is `function_ref<R(A...) noexcept(E)>`.



§ 5.2. Option B — Renaming `std::nontype` to `std::constant_arg`

In [\[version.syn\]](#), update the feature-test macro:



```
#define __cpp_lib_function_ref 202306L 20XXXXL // also in <functional>
```

In [\[utility.syn\]](#), modify the header synopsis as follows:



```
// nontype constant argument tag
template<auto V>
struct nontype_t constant_arg_t {
    explicit nontype_t() = default;
};
template<auto V> constexpr nontype_t<V> nontype{};
template<auto V> constexpr constant_arg_t<V> constant_arg;
```

In [\[func.wrap.ref.class\]](#), replace every occurrence of `nontype_t` with `constant_arg_t`:



```
namespace std {
    template<class R, class... ArgTypes>
    class function_ref<R(ArgTypes...) cv noexcept(noex)> {
    public:
        // [func.wrap.ref.ctor], constructors and assignment operators
        template<class F> function_ref(F*) noexcept;
        template<class F> constexpr function_ref(F&&) noexcept;
        template<auto f> constexpr function_ref(nontype_t constant_arg_t<f>) noexcept;
        template<auto f, class U> constexpr function_ref(nontype_t constant_arg_t<f>)
        template<auto f, class T> constexpr function_ref(nontype_t constant_arg_t<f>)

        constexpr function_ref(const function_ref&) noexcept = default;
        constexpr function_ref& operator=(const function_ref&) noexcept = default;
        template<class T> function_ref& operator=(T) = delete;

        // [func.wrap.ref.inv], invocation
        R operator()(ArgTypes...) const noexcept(noex);

    private:
        template<class... T>
            static constexpr bool is-invocable-using = see below; // exposition only

        R (*thunk_ptr)(BoundEntityType, Args&&...) noexcept(noex); // exposition only
        BoundEntityType bound-entity; // exposition only
    };

    // [func.wrap.ref.deduct], deduction guides
    template<class F>
```

```

function_ref(F*) -> function_ref<F>;
template<auto f>
function_ref(nontype_t constant_arg_t<f>) -> function_ref<see below>;
template<auto f, class T>
function_ref(nontype_t constant_arg_t<f>, T&&) -> function_ref<see below>;
}

```



In `[func.wrap.ref.ctor]`, replace every occurrence of `nontype_t` with `constant_arg_t`:



```

[...]
template<auto f> constexpr function_ref(nontype_t constant_arg_t<f>) noexcept;
[...]
template<auto f, class U>
constexpr function_ref(nontype_t constant_arg_t<f>, U&& obj) noexcept;
[...]
template<auto f, class T>
constexpr function_ref(nontype_t constant_arg_t<f>, cv T* obj) noexcept;
[...]
template<class T> function_ref& operator=(T) = delete;

```

21 Constraints:

- T is not the same type as `function_ref`,
- `is_pointer_v<T>` is `false`, and
- T is not a specialization of `nontype_t constant_arg_t`.

In `[func.wrap.ref.deduct]`, replace every occurrence of `nontype_t` with `constant_arg_t`:



```

[...]
template<auto f>
function_ref(nontype_t constant_arg_t<f>) -> function_ref<see below>;
[...]
template<auto f, class T>
function_ref(nontype_t constant_arg_t<f>, T&&) -> function_ref<see below>;
[...]
```

§ 6. References

[N5008] *Thomas Köppe*. Working Draft, Programming Languages — C++ (2025-03-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf>



[P0792R14] *Vittorio Romeo, Zhihao Yuan, Jarrad Waterloo*. `function_ref`: a type-erased callable reference (2022-02-08)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p0792r14.html>

[P1045R1] *David Stone*. `constexpr` Function Parameters (2019-09-27)
<https://open-std.org/JTC1/SC22/WG21/docs/papers/2019/p1045r1.html>

[P2472R3] *Jarrad J. Waterloo, Zhihao Yuan*. `make_function_ref` more functional (2022-05-12)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2472r3.html>

[P2781R8] *Hana Dusíková, Matthias Kretz, Zach Laine*. `std::constant_wrapper` (2025-03-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2781r8.html>

[P2841R1] *Corentin Jabot*. Concept and variable-template `template-parameters` (2023-10-14)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2841r1.pdf>

[P3312R1] *Bengt Gustafsson*. Overload Set Types (2025-04-16)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3312r1.pdf>

[GitHub] *Jan Schultke*. Reference implementation of `std::function_ref` with `std::stateless` (2025-06-14)
https://github.com/Eisenwave/nontype_functional